SOBEK: STREAMING EQUIVARIANT TENSOR PRODUCT CONVOLUTIONS

Vladimir Chorošajev
Cortex Discovery
vladimir.chorosajev@cortexdiscovery.com

Cédric Bény
Cortex Discovery
cedric.beny@cortexdiscovery.com

July 20, 2026

arXiv:2607.18074v1 [cs.LG] 20 Jul 2026

ABSTRACT

Equivariant graph neural networks repeatedly apply edge-conditioned tensor-product convolutions over graph edges. Conventional implementations materialize edge-specific weights, messages, and adjoints, causing tensor-product workspace and memory traffic to grow rapidly with graph size and operator width. This limits feasible workloads and can prevent larger problems from fully utilizing the GPU.

We show that these edge-sized intermediates are artifacts of the execution schedule, not requirements of the equivariant operator. By reassociating radial projection, spherical-harmonic coupling, and graph aggregation, edge-local products can be consumed directly into bounded receiver-side state. The resulting streaming formulation preserves fully connected multiplicity mixing and extends through forward, backward, and double backward.

We implement this formulation in SOBEK[1], a generated-CUDA backend, and evaluate it across edge-scaling regimes and varied feature structures. Across two operator families and all three differentiation orders, SOBEK is faster in all 75 capacity-matched comparisons, with speedups ranging from $1.2\times$ to $49.7\times$, and reduces peak allocated memory by up to 99%. It also executes workloads up to two orders of magnitude beyond OpenEquivariance's capacity while retaining near-peak throughput. These results show that edge-scaled tensor-product workspace is a property of the conventional schedule, not of equivariant convolution itself.

1 Introduction

Geometric deep learning encodes symmetries in model layers, extending translation- and permutation-aware architectures to rotations, reflections, and translations in three dimensions [1]. Equivariant models have been successfully applied to spherical images (Clebsch–Gordan Nets [2]), point clouds and N-body simulation (SE(3)-Transformers [3]), and computational physics (SEGNNs [4]).

Among equivariant architectures, one influential family represents node features as irreducible representations (*irreps*), distances through radial basis embeddings, and directions through spherical harmonics. Clebsch–Gordan tensor products combine node features with radial and angular edge features while preserving their prescribed transformations. Tensor Field Networks established this construction for point-cloud convolutions [5]; we refer to the resulting operator primitive as *edge-conditioned geometric convolution*.

This operator primitive has become especially prominent in chemical machine learning, where machine-learned interatomic potentials must predict invariant energies and equivariant forces from expensive reference data. NequIP demonstrated the data efficiency and accuracy of symmetries as inductive bias [6]; MACE, Equiformer, and Allegro likewise use this geometric primitive for molecular and materials modeling [7, 8, 9].

The operator is computationally and memory intensive. Each edge couples sender features, spherical harmonics, and edge-specific weights across tensor-product paths. Cost grows with edge count: wider representations enlarge channel

[1] Source: https://github.com/Cortex-Discovery/sobek

contractions, higher angular order adds coupling paths and component arithmetic, and larger batches and cutoffs multiply edges. Tensor-product convolution can therefore dominate training and inference, forcing tradeoffs among width, angular order, interaction range, and batch size.

This cost can constrain not only model scale, but also the expressivity of the geometric convolution itself. Practical architectures may restrict how input and output channels interact to keep edge-conditioned weights tractable [8]. These channelwise or depthwise operators can be effective, but they represent a narrower family of edge-conditioned equivariant maps than fully connected mixing [10].

Systems such as OpenEquivariance [11] and NVIDIA cuEquivariance [12] accelerate tensor-product evaluation through specialized kernels. Architectural approaches such as EquiformerV2 [13] instead reduce the arithmetic cost of the angular operator. The present work addresses a different question: the dataflow of the complete edge-conditioned convolution.

A conventional decomposition first expands each graph edge embedding into a vector of tensor-product weights, applies the tensor product to form an edge message, and finally aggregates messages at receiver nodes. The expanded weights and messages are single-use intermediates, yet their storage scales with the number of edges. They can therefore dominate peak memory and memory traffic for wide representations or high-density geometric graphs.

We show that these edge-sized tensors are scheduling artifacts, derive a streaming schedule by reassociating the contractions as receiver-owned reductions, and introduce SOBEK as a generated-CUDA backend that implements this schedule for edge-conditioned equivariant convolutions. SOBEK never materializes complete edge-weight or edge-message tensors while preserving the operator's forward, backward, and double-backward semantics. The reformulation is algebraically equivalent to the conventional schedule, and every reported measurement passes strict double-peer numerical validation. Only inherent graph and feature storage remains edge-scaled; tensor-product workspace is bounded by a fixed scratch budget. The resulting implementation also delivers higher sustained throughput and permits larger workloads to occupy the GPU more fully.

This work makes three contributions:

- **Algorithm:** We identify per-edge weights, messages, and adjoints as scheduling artifacts and derive algebraically equivalent streamed forward, backward, and double backward with bounded tensor-product workspace.
- **System:** We provide an implementation of the streaming schedule in SOBEK, a deterministic CUDA code-generation backend supporting fully connected `uvw` and channelwise `uvu` execution across all three differentiation orders without edge-weight workspace.
- **Evidence:** Across controlled scaling sweeps and 25 feature profiles, SOBEK is faster in all 75 capacity-matched comparisons, reduces peak allocation by up to 99%, and sustains near-peak throughput at workloads far beyond OpenEquivariance's capacity.

2 Background

2.1 Geometric equivariance

Geometric equivariance specifies how model outputs transform when the input geometry is transformed. Invariant outputs remain unchanged, while equivariant outputs follow a prescribed transformation law. Molecular energy prediction provides a concrete example: for a rotation matrix R and atomic positions $\mathbf{r}$, energy remains unchanged while forces rotate with the positions:

$$E(R\mathbf{r}) = E(\mathbf{r}), \qquad \mathbf{F}(R\mathbf{r}) = R\mathbf{F}(\mathbf{r}). \tag{1}$$

More generally, let π_g and π'_g describe how a transformation g acts respectively on the input and output of a map f. We say that f is equivariant when

$$f(\pi_g(x)) = \pi'_g(f(x)). \tag{2}$$

Linear actions of the group of improper rotations $\mathrm{O}(3)$ can be expressed as a direct sum of irreducible representations (irreps). Here we consider the case where f is a bilinear map, and π_g is a tensor product of linear representations. Any such equivariant f is a linear combination of basic operations (paths) characterized by the Clebsch-Gordon coefficients, and mapping specific input irreps into a specific output irrep. In geometric GNNs, each edge of the graph is associated to such an operation, with edge-dependent path weights [5, 14].

2.2 Irreducible features and tensor products

An equivariant feature field is a direct sum of irreducible representations,

$$x = \bigoplus_{\ell,\sigma} x_{\ell,\sigma}, \qquad x_{\ell,\sigma} \in \mathbb{R}^{C_{\ell,\sigma} \times (2\ell+1)}, \tag{3}$$

where ℓ is the angular order, $\sigma \in \{-1, 1\}$ is parity, and $C_{\ell,\sigma}$ is the multiplicity of that irrep. The group acts on the $2\ell + 1$ angular components, while multiplicities form a learned channel axis.

The flattened feature dimension is

$$C_x = \sum_{\ell,\sigma} C_{\ell,\sigma}(2\ell + 1). \tag{4}$$

Tensor-product layers combine an input irrep with a spherical-harmonic irrep. A valid output satisfies the Clebsch–Gordan selection rules

$$\ell_{\text{out}} \in |\ell_{\text{in}} - \ell_{\text{sh}}|, \ldots, \ell_{\text{in}} + \ell_{\text{sh}}, \qquad \sigma_{\text{out}} = \sigma_{\text{in}}\sigma_{\text{sh}}. \tag{5}$$

Each valid combination defines a tensor-product path. Clebsch–Gordan coefficients determine how its angular components couple, while learned weights act only on multiplicities.

2.3 Channelwise and fully connected multiplicity mixing

Tensor-product connection modes specify how features are mixed along the multiplicity axis for a path allowed by Clebsch-Gordan selection rules [14, 15]. Let u, v, and w index the input, spherical-harmonic, and output multiplicities of one valid angular path. For an edge e with unit direction $\widehat{\mathbf{r}}$, a fully connected `uvw` instruction computes

$$z_w = \sum_{u,v} \omega_{uvw}(e) \left[x_u \otimes_{\text{CG}} Y_v(\widehat{\mathbf{r}}) \right], \tag{6}$$

and therefore uses $C_{\text{in}} \cdot C_{\text{sh}} \cdot C_{\text{out}}$ edge-dependent weights for that path. In the channelwise `uvu` mode, the repeated index ties the output multiplicity to the first input multiplicity,

$$z_u = \sum_{v} \omega_{uv}(e) \left[x_u \otimes_{\text{CG}} Y_v(\widehat{\mathbf{r}}) \right], \tag{7}$$

so the instruction requires $C_{\text{out}} = C_{\text{in}}$ and only $C_{\text{in}}C_{\text{sh}}$ weights. This equality is local to one instruction; different irrep blocks may still have different multiplicities, and later equivariant linear layers may mix their outputs.

These modes differ in their expressivity. A fully connected `uvw` path allows the invariant edge condition—in molecular models, typically a function of interatomic distance—to select a different mixing between every input and output multiplicity channel. A `uvu` path can still modulate each tied channel with distance, but it cannot change which input and output channels are coupled within that path. Surrounding equivariant linear layers can still produce expressive networks, but they do not make one `uvu` instruction equivalent to a general `uvw` instruction at fixed width. This operator-level expressivity distinction was studied by Xie et al. [10].

2.4 Edge-conditioned tensor product convolution

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed graph with source node j and receiver node i on edge $j \to i$. For node positions $\mathbf{r}_i$, define the relative displacement, distance, and direction as

$$\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i, \qquad r_{ij} = \|\mathbf{r}_{ij}\|, \qquad \widehat{\mathbf{r}}_{ij} = \frac{\mathbf{r}_{ij}}{r_{ij}}. \tag{8}$$

The angular input Y_{ij} contains spherical harmonics of $\widehat{\mathbf{r}}_{ij}$, while $\phi_{ij} \in \mathbb{R}^K$ is a radial or more general invariant edge embedding.

Let $\mathcal{P}$ be the set of valid tensor-product paths. Each path $\pi \in \mathcal{P}$ selects an input irrep, a spherical-harmonic irrep, an output irrep, and their Clebsch–Gordan coupling. Because the spherical harmonics have multiplicity one, path π has

$$n_\pi = C_\pi^{\text{in}} C_\pi^{\text{out}}, \tag{9}$$

where C_π^{in} and C_π^{out} are the coupled input and output multiplicities. The index $a \in \{1, \ldots, n_\pi\}$ below is a flattened input/output multiplicity pair.

The total number of edge-specific tensor-product weights is

$$N_{\mathrm{w}} = \sum_{\pi \in \mathcal{P}} n_\pi. \tag{10}$$

An edge-conditioned equivariant convolution computes

$$m_i = \sum_{j \in \mathcal{N}(i)} \sum_{\pi \in \mathcal{P}} \sum_{a=1}^{n_\pi} W_{ij,\pi,a}\, T_{\pi,a}\left(x_j, Y_{ij}\right), \tag{11}$$

where $T_{\pi,a}(x_j, Y_{ij})$ is the angular tensor-product contribution for path π and multiplicity pair a [6, 7, 14], and C_{out} denotes the flattened output feature dimension.

The edge-dependent weights are generated by a learned radial projection:

$$W_{ij,\pi,a} = \sum_{q=1}^{K} \phi_{ij,q} A_{q,\pi,a}. \tag{12}$$

After flattening (π, a), the three tensors have shapes

$$\phi \in \mathbb{R}^{|\mathcal{E}| \times K}, \qquad A \in \mathbb{R}^{K \times N_{\mathrm{w}}}, \qquad W \in \mathbb{R}^{|\mathcal{E}| \times N_{\mathrm{w}}}, \qquad W = \phi A. \tag{13}$$

The model stores A once per layer, and the graph supplies ϕ once per edge. The tensor W is neither a model parameter nor an input; it is a transient expansion produced by the radial projection and consumed by the tensor-product contraction.

2.5 Memory cost of materialized execution

A conventional materialized evaluation forms both the edge weights and the edge messages before reducing them to the node output $m \in \mathbb{R}^{|\mathcal{V}| \times C_{\mathrm{out}}}$. For a scalar type with b bytes, the edge-weight tensor W and edge-message tensor $M \in \mathbb{R}^{|\mathcal{E}| \times C_{\mathrm{out}}}$ require $b|\mathcal{E}|N_{\mathrm{w}}$ and $b|\mathcal{E}|C_{\mathrm{out}}$ bytes, respectively.

The graph inputs ϕ and Y are also edge-sized, and any exact execution must either store or recompute them. Their widths are K and $\dim(\mathcal{R}_{\mathrm{sh}})$, while materialization expands each edge to N_{w} weights and a C_{out}-dimensional message. The avoidable cost is therefore not edge dependence itself, but the much wider intermediates created from those inputs.

Both factors in $|\mathcal{E}|N_{\mathrm{w}}$ can grow. At fixed angular structure, scaling every input and output multiplicity by s gives

$$N_{\mathrm{w}}^{\mathtt{uvw}}(sC) = s^2 N_{\mathrm{w}}^{\mathtt{uvw}}(C), \tag{14}$$

because each path connects every input multiplicity to every output multiplicity. The edge count satisfies $|\mathcal{E}| = |\mathcal{V}|\bar{d}$, where $\bar{d}$ is the average directed degree. Increasing the number of nodes or their average degree therefore increases the materialized cost, which reaches $O(|\mathcal{V}|^2 N_{\mathrm{w}})$ in the dense limit.

For three-dimensional radius graphs at fixed particle density ρ, $|\mathcal{E}| \propto |\mathcal{V}|\rho r_{\mathrm{c}}^3$, where r_{c} is the interaction cutoff. Work and materialized edge memory therefore grow cubically with the cutoff. Table 1 illustrates how increasing multiplicities expand materialized edge-weight memory at fixed graph size.

Table 1: Edge-specific tensor-product weights for equal-width profiles at one million directed edges. Counts assume $L_{\max} = 2$, matching input/output irreps, one spherical-harmonic copy per angular order, and fully connected multiplicity mixing. Profile entries list multiplicities in ascending angular order.

Profile	N_{w}	fp32 M_W at 1.0×10^6 edges (GiB)
32/32/32	11,264	42.0
64/64/64	45,056	167.8
128/128/128	180,224	671.4
256/256/256	720,896	2,685.5

This memory cost is not inherent to equivariant convolution. The next section shows that the same operator, including its first and second derivatives, can be evaluated without ever assembling the complete edge-weight or edge-message tensors.

3 Streaming Execution Schedule

Streaming is a property of intermediate lifetime, not a particular kernel architecture. The materialized evaluation order creates complete tensors W and M in memory, that the convolution does not require. A streaming schedule instead consumes their values incrementally without retaining the complete edge-sized tensors as global intermediates. This does not require a single fused kernel, zero scratch, a specific graph ordering, or a particular reduction strategy.

3.1 Eliminating materialized edge weights

Substituting the radial projection in Eq. 12 into Eq. 11 gives

$$m_i = \sum_{j \in \mathcal{N}(i)} \sum_{\pi \in \mathcal{P}} \sum_{a=1}^{n_\pi} \left(\sum_{q=1}^{K} \phi_{ij,q} A_{q,\pi,a} \right) T_{\pi,a}(x_j, Y_{ij}). \tag{15}$$

The equivalent streaming order is

$$m_i = \sum_{j \in \mathcal{N}(i)} \sum_{\pi \in \mathcal{P}} \sum_{a=1}^{n_\pi} \sum_{q=1}^{K} \phi_{ij,q} A_{q,\pi,a} T_{\pi,a}(x_j, Y_{ij}). \tag{16}$$

The two expressions are identical in exact arithmetic. Both use the same paths, radial model, and Clebsch–Gordan coefficients to define the same equivariant operator. Only the lifetime of intermediate values changes. Their accumulation orders may differ in floating-point arithmetic, so numerical comparisons should use an explicit tolerance.

Define the edge-specific scalar weight as

$$w_{ij,\pi,a} = \sum_{q=1}^{K} \phi_{ij,q} A_{q,\pi,a} \tag{17}$$

A streaming schedule need only keep this scalar live until its tensor-product contribution has been consumed. The full collection $W \in \mathbb{R}^{|\mathcal{E}| \times N_{\mathrm{w}}}$ is therefore an artifact of one evaluation order, not a mathematical input or output of the convolution. Applying the same lifetime rule to the contribution $w_{ij,\pi,a} T_{\pi,a}(x_j, Y_{ij})$ also avoids a complete edge-message tensor.

The lifetime argument is independent of the angular tensor-product algorithm and multiplicity connection mode: those choices determine arithmetic and channel connectivity, while streaming determines whether their edge-conditioned values are retained across the complete graph.

3.2 Backward without an edge-weight-adjoint tensor

Let $\bar{m}_i$ denote the output adjoint and define the edge-local scalar

$$h_{ij,\pi,a} = \langle \bar{m}_i, T_{\pi,a}(x_j, Y_{ij}) \rangle \,. \tag{18}$$

The radial-parameter and edge-embedding adjoints are then

$$\bar{A}_{q,\pi,a} = \sum_{i} \sum_{j \in \mathcal{N}(i)} \phi_{ij,q} h_{ij,\pi,a}, \tag{19}$$

$$\bar{\phi}_{ij,q} = \sum_{\pi,a} A_{q,\pi,a} h_{ij,\pi,a}. \tag{20}$$

For the tensor-product operands, let $D_x T^*_{\pi,a}$ and $D_Y T^*_{\pi,a}$ denote the adjoints of the two partial Jacobians. Their gradients are

$$\bar{x}_j = \sum_{i:\, j\in\mathcal{N}(i)} \sum_{\pi,a} w_{ij,\pi,a} D_x T_{\pi,a}(x_j, Y_{ij})^* \bar{m}_i, \tag{21}$$

$$\bar{Y}_{ij} = \sum_{\pi,a} w_{ij,\pi,a} D_Y T_{\pi,a}(x_j, Y_{ij})^* \bar{m}_i. \tag{22}$$

Every quantity on the right-hand sides can be formed for one edge and path, accumulated into its destination, and discarded. The scalar $h_{ij,\pi,a}$ equals the corresponding entry of the edge-weight adjoint but can be consumed immediately, so the complete tensor $\bar{W} \in \mathbb{R}^{|\mathcal{E}|\times N_{\mathrm{w}}}$ is never needed. First backward therefore admits the same streaming schedule as forward.

3.3 Double backward by differentiating the streamed contractions

The same locality extends to second order. Square brackets below denote application of a partial Jacobian to a direction, not indexing: $D_x T_{\pi,a}[\dot{x}_j]$ is the directional derivative with respect to x_j along $\dot{x}_j$, and analogously for $D_Y T_{\pi,a}[\dot{Y}_{ij}]$.

For directions $\dot{x}$, $\dot{Y}$, $\dot{\phi}$, and $\dot{A}$, define

$$\dot{w}_{ij,\pi,a} = \sum_q \left(\dot{\phi}_{ij,q} A_{q,\pi,a} + \phi_{ij,q} \dot{A}_{q,\pi,a} \right), \tag{23}$$

$$\dot{T}_{ij,\pi,a} = D_x T_{\pi,a}[\dot{x}_j] + D_Y T_{\pi,a}[\dot{Y}_{ij}]. \tag{24}$$

The forward directional output and the directional edge-weight adjoint are

$$\dot{m}_i = \sum_{j\in\mathcal{N}(i)} \sum_{\pi,a} \left(\dot{w}_{ij,\pi,a} T_{\pi,a}(x_j, Y_{ij}) + w_{ij,\pi,a} \dot{T}_{ij,\pi,a} \right), \tag{25}$$

$$\dot{h}_{ij,\pi,a} = \langle \dot{\bar{m}}_i, T_{\pi,a}(x_j, Y_{ij}) \rangle + \left\langle \bar{m}_i, \dot{T}_{ij,\pi,a} \right\rangle. \tag{26}$$

Both depend only on edge-local values. Differentiating the first-order adjoints applies the same product rule; for example,

$$\dot{\bar{A}}_{q,\pi,a} = \sum_i \sum_{j\in\mathcal{N}(i)} \left(\dot{\phi}_{ij,q} h_{ij,\pi,a} + \phi_{ij,q} \dot{h}_{ij,\pi,a} \right), \tag{27}$$

$$\dot{\bar{\phi}}_{ij,q} = \sum_{\pi,a} \left(\dot{A}_{q,\pi,a} h_{ij,\pi,a} + A_{q,\pi,a} \dot{h}_{ij,\pi,a} \right). \tag{28}$$

For the operand adjoints, define the local contractions

$$B^x_{ij,\pi,a} = D_x T^*_{\pi,a} \bar{m}_i, \qquad B^Y_{ij,\pi,a} = D_Y T^*_{\pi,a} \bar{m}_i, \tag{29}$$

and let $\dot{B}^x$ and $\dot{B}^Y$ denote their directional derivatives. The remaining second adjoints are

$$\dot{\bar{x}}_j = \sum_{i:\, j\in\mathcal{N}(i)} \sum_{\pi,a} \left(\dot{w}_{ij,\pi,a} B^x_{ij,\pi,a} + w_{ij,\pi,a} \dot{B}^x_{ij,\pi,a} \right), \tag{30}$$

$$\dot{\bar{Y}}_{ij} = \sum_{\pi,a} \left(\dot{w}_{ij,\pi,a} B^Y_{ij,\pi,a} + w_{ij,\pi,a} \dot{B}^Y_{ij,\pi,a} \right). \tag{31}$$

Because the tensor product is bilinear, $\dot{B}^x$ and $\dot{B}^Y$ use the same edge operands and Clebsch–Gordan coefficients as the forward and first-order contractions. The complete second-order program therefore remains edge-local and requires no materialized W, $\bar{W}$, directional edge-weight tensor, edge-message tensor, or dense Hessian.

3.4 Global lifetime summary

Table 2 summarizes the complete global tensors used by materialized and streaming schedules. It does not constrain local or bounded workspace within a particular implementation.

Table 2: Complete global tensors used by materialized and streaming tensor-product convolution schedules.

Quantity	Shape	Materialized	Streaming
Node features	$\lvert\mathcal{V}\rvert \times C_{\text{in}}$	stored	stored
Edge embeddings	$\lvert\mathcal{E}\rvert \times K$	stored	stored
Spherical harmonics	$\lvert\mathcal{E}\rvert \times \dim(\mathcal{R}_{\text{sh}})$	stored or recomputed	stored or recomputed
Edge TP weights	$\lvert\mathcal{E}\rvert \times N_{\text{w}}$	materialized	not required as a complete tensor
Edge messages	$\lvert\mathcal{E}\rvert \times C_{\text{out}}$	materialized or partially materialized	not required as a complete tensor
Node outputs	$\lvert\mathcal{V}\rvert \times C_{\text{out}}$	stored	stored

The transformation is therefore conceptually independent of data layout, precision, tensor-product instruction representation, and hardware mapping. The next section shows how this implementation-independent principle can be realized as a practical execution architecture without weakening the edge-tensor lifetime guarantee.

4 SOBEK Design and Implementation

4.1 Execution overview

Section 3 established that complete edge-weight tensors are not required by the operator or its derivatives. SOBEK realizes that lifetime property by ending edge-sized execution inside receiver traversal: edge-local contractions reduce directly into bounded receiver-side state, after which channel-space operations act on receiver rows. Neither W, an edge-message tensor, nor $\bar{W}$ crosses a kernel boundary as a global tensor.

Here, *receiver-factorized* means that work is organized around each destination node rather than around a materialized edge tensor. For every receiver and output group, contributions from its incoming edges first reduce into a bounded state S_g that contains the angular, radial, and input-channel dimensions but no output-channel dimension. The learned output-channel mixing is then applied once to that receiver state. This separates graph traversal from channel mixing and prevents either stage from producing a complete edge-weight or edge-message array.

Figure 1 summarizes the receiver-factorized forward and backward dataflow for one output group g. It is an algebraic view rather than a kernel-launch trace. Let $E = |\mathcal{E}|$ and $N = |\mathcal{V}|$; R_g counts receiver/output-component rows, and $D_g = \sum_{\pi \in \mathcal{P}_g} KC_\pi^{\text{in}}$ is the flattened path, radial, and input-multiplicity width.

The edge-local CG map combines ϕ_{ij}, x_j, and Y_{ij} into s_{ij}. Receiver gather forms S_g, and channel mixing produces $m_g = S_g A_g$. Backward reverses this flow: channel mixing forms $\bar{S}_g$, receiver scatter supplies edge-local adjoints, and deterministic source gather performs the cross-edge reduction for $\bar{x}$. The following subsections give the corresponding contractions and generated execution.

4.2 Receiver-factorized forward

SOBEK realizes conceptual streaming by reassociating the contraction around each receiver. Let g denote an output-irrep group and $\mathcal{P}_g$ the paths that contribute to it. For a path π, input multiplicity u, and output-irrep component m_o, define the unweighted angular contraction

$$C_{\pi,u,m_o}(x_j, Y_{ij}) = \sum_{m_i,m_y} c^{\pi}_{m_i,m_y,m_o} x_{j,\pi,u,m_i} Y_{ij,\pi,m_y}, \tag{32}$$

where $c^{\pi}_{m_i,m_y,m_o}$ contains the path normalization and sparse Clebsch–Gordan coefficient. A receiver-contiguous traversal accumulates

$$S_{i,\pi,q,u,m_o} = \sum_{j \in \mathcal{N}(i)} \phi_{ij,q} C_{\pi,u,m_o}(x_j, Y_{ij}). \tag{33}$$

For a fully connected `uvw` instruction, channel mixing then gives

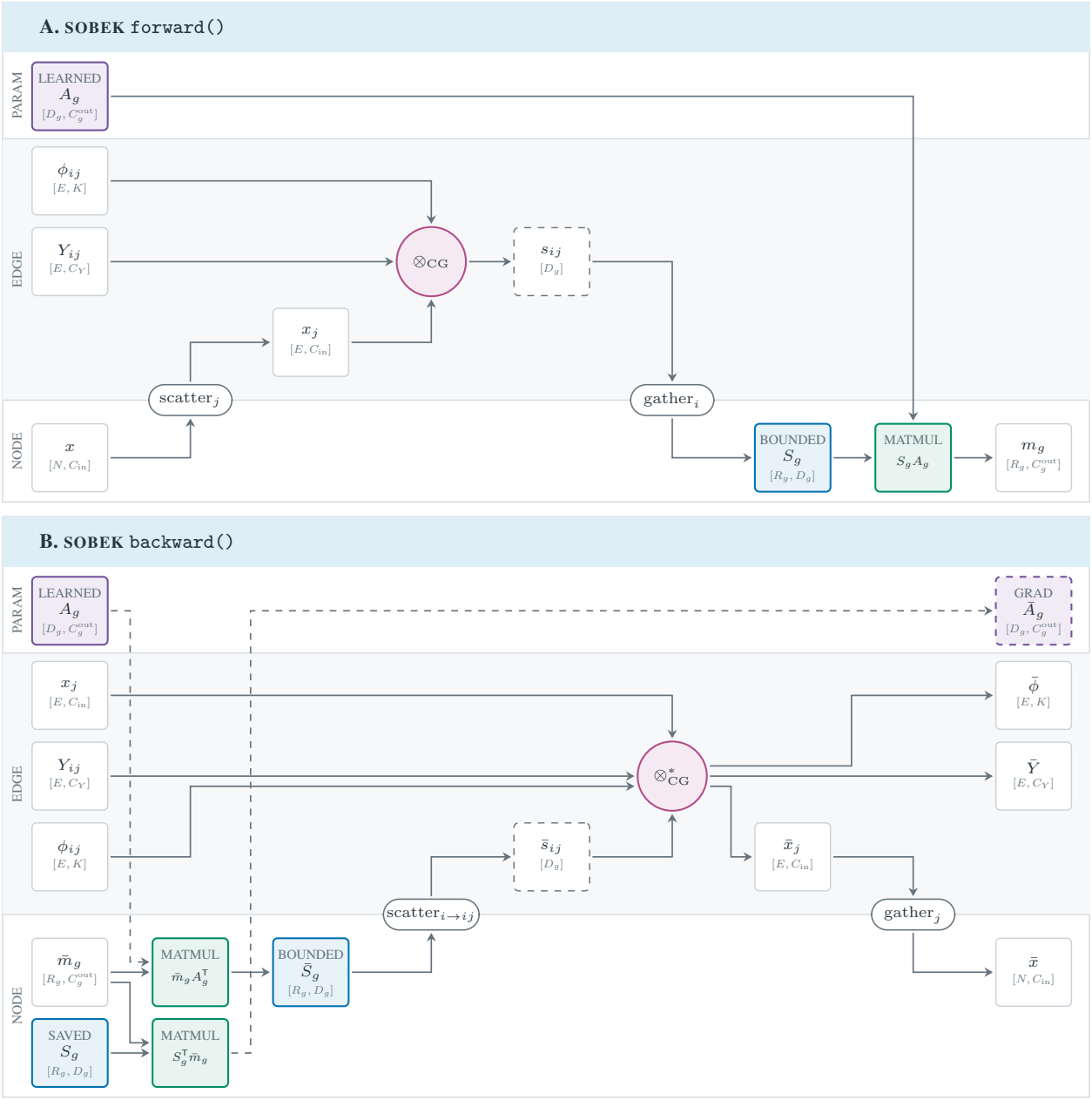


Figure 1: Receiver-factorized SOBEK dataflow for one output group. Lanes distinguish parameter-, edge-, and node-indexed values. Purple, blue, green, and magenta mark learned parameters, bounded receiver state, channel matrix products, and edge-local CG operations, respectively; rounded scatter/gather nodes change graph ownership. Dashed gray boxes are edge-local transient values, while the dashed purple box is the accumulated parameter gradient. No complete edge-weight, edge-message, or edge-weight-adjoint tensor appears.

$$m^{(g)}_{i,w,m_o} = \sum_{\pi \in \mathcal{P}_g} \sum_{q=1}^{K} \sum_{u=1}^{C^{\text{in}}_\pi} S_{i,\pi,q,u,m_o} A_{q,\pi,u,w}. \tag{34}$$

For each output group, SOBEK packs the receiver and output-irrep component indices as rows, the radial and input-multiplicity indices as the contracted dimension, and the output multiplicity as columns. Equation 34 is then the channel matrix product

$$m = SA. \tag{35}$$

Equations 33–35 are an exact reassociation of Eq. 16. The receiver state has no output-multiplicity axis and does not scale with the number of edges. A generated receiver traversal constructs S, a channel GEMM applies A, and a compact write kernel places the result in the output tensor. Forward states needed for radial-parameter gradients are retained only as complete output groups within the fixed scratch budget; other groups are recomputed over bounded receiver ranges. This policy changes reuse and recomputation, not the streaming transformation itself.

4.3 Receiver-factorized backward

Backward differentiates the same receiver program. With receiver and output-irrep component indices packed as the row dimension of Eq. 35, the two channel-space adjoints are

$$\bar{S} = \bar{m} A^{\mathsf{T}}, \qquad \bar{A} = S^{\mathsf{T}} \bar{m}. \tag{36}$$

Receiver/channel GEMMs form $\bar{S}$ and accumulate $\bar{A}$. Generated edge-owned kernels differentiate Eq. 33 to form $\bar{\phi}$ and $\bar{Y}$ in receiver-sorted edge order; these quantities have a unique output location per edge and require no cross-edge reduction. A separate source-contiguous CSR traversal owns $\bar{x}$: one source row has one fixed reduction order, avoiding both atomics and nondeterministic accumulation. High-work receiver recomputation uses the same grouped output-component traversal as forward, while edge and source kernels assign disjoint edge or channel ownership so that their reductions remain local.

Compatible $\bar{S}$ states share a source traversal when they fit the scratch bound; otherwise SOBEK chunks output groups and receiver ranges while keeping retained receiver-side state under the same fixed budget. These choices change reuse and recomputation, not the adjoint equations or reduction ownership. The detailed fit-aware source policy is given in Appendix B.2.

Crucially, backward does not form an edge-weight adjoint $\bar{W}$ and then consume it incrementally. Radial, angular, and node-feature gradients are obtained directly from S, $\bar{S}$, and the graph traversals, so $\bar{W}$ is never constructed, not even as an edge chunk.

4.4 Double backward as the differentiated receiver program

Double backward is differentiation of Eqs. 33–36, rather than an unrelated collection of higher-order kernels. A receiver traversal constructs the primal state S and its active directional state dS. The corresponding channel identities are

$$dm = dS\, A + S\, dA, \tag{37}$$

$$d\bar{S} = d\bar{m}\, A^{\mathsf{T}} + \bar{m}\, dA^{\mathsf{T}}, \tag{38}$$

$$d\bar{A} = dS^{\mathsf{T}} \bar{m} + S^{\mathsf{T}} d\bar{m}. \tag{39}$$

GEMMs construct output tangents, adjoints, and radial-parameter derivatives from the same bounded receiver states. Forward-retained S is reused, while the additional primal, adjoint, and directional states follow the same fit-and-recompute policy as first backward. Receiver CSR owns edge-local Hessian terms for ϕ and Y, and source CSR owns deterministic node adjoints. Thus the receiver, edge, and source ownership extends through second order without a separate kernel architecture or edge-scaled state; Appendix B.2 summarizes the shared scratch rule.

This organization builds directly on the higher-order decomposition developed by the OpenEquivariance authors [11], expressing double backward as forward-style and backward-style replay terms and lowers those terms into dedicated receiver- and source-major kernels. SOBEK lowers the same mathematical decomposition into the receiver program above.

4.5 Generated deterministic runtime and execution contracts

SOBEK accepts input, output, and spherical-harmonic irrep structures and generates specialized kernels for forward, backward, and double backward.

A thin Python/PyTorch wrapper validates the contract, adapts layout, prepares the graph, and connects generated entry points to autograd. Following OpenEquivariance [11], SOBEK treats the static tensor-product description as input to a kernel generator. It enumerates valid paths and sparse Clebsch–Gordan terms and emits CUDA source code with compile-time paths, loop bounds, offsets, and coefficients. PyTorch compiles and caches the extension, so steady-state execution contains neither an instruction interpreter nor code-generation overhead.

SOBEK compiles channelwise `uvu` directly rather than pruning off-diagonal mixing from dense `uvw`. Its deterministic forward, backward, and double-backward schedule preserves tied-channel semantics without edge workspace.

Generated kernels use `ir_mul` layout, with adjacent multiplicity channels contiguous within each irrep block. The wrapper transposes the default `mul_ir` layout at the boundary, or callers can retain `ir_mul` across layers. Cached stable CSR views give receiver and source reductions fixed ownership without nondeterministic atomics. CUDA Graph capture and TF32 channel GEMMs are optional; all reported comparisons use eager fp32 with TF32 disabled. Appendix B provides additional CUDA implementation details, including compile-time specialization, kernel grouping, and runtime scratch-budget policy.

5 Evaluation

Our evaluation asks three questions:

1. **Memory scaling.** How does peak memory scale with edge count and operator width?
2. **Throughput scaling.** Does throughput improve as larger workloads fill the GPU under both degree and graph-count scaling?
3. **Generality across feature structures.** Does the approach preserve its correctness, memory, and throughput advantages across varied node-feature representations, including fully connected `uvw` and channelwise `uvu`?

Forward, full first backward, and native double backward are included in every sweep and comparison rather than treating differentiation order as a separate question.

Answering all three questions requires a common reference for scaling, throughput, and numerical equivalence. Native e3nn [14] is a general-purpose reference, while cuEquivariance [12] also provides a fused fully connected tensor-product convolution with graph gather/scatter. We choose OpenEquivariance [11] as the optimized reference because it provides the directly matched deterministic forward, backward, and double-backward contract required by our comparison and validation protocol.

5.1 Reference and experimental design

Comparison and validation policy. We compare both backends on an NVIDIA RTX 5090 using fp32 throughout. Every presented pair receives the same complete input: graph topology and ordering, features, weights, and all other tensors. All SOBEK measurements use a 128 MiB receiver scratch budget and highest-precision fp32 channel GEMMs. Compilation, graph generation, and graph preparation are outside timed execution.

Every data point presented in this paper passes strict double-peer validation: two SOBEK evaluations match exactly, two OpenEquivariance evaluations match exactly, and all four agree within the cross-backend tolerance. The base tolerance is $\mathrm{rtol} = \mathrm{atol} = 5 \times 10^{-4}$; large or dense reductions use $\mathrm{rtol} = 2 \times 10^{-3}$ and $\mathrm{atol} = 2 \times 10^{-2}$. These tolerances accommodate changes in the order of floating-point additions between the two schedules.

Each scaling point runs in a fresh process. After at least five calls and 50 ms of warm-up, CUDA-event timing collects 10–30 samples totaling at least 250 ms and reports the median. Peak allocated CUDA memory is measured after compilation warm-up in the same isolated worker. A confirmed OOM is recorded at the attempted graph size, and no later point is fabricated for that backend.

Scaling axes. For *degree scaling*, we construct radius graphs on one set of positions for 2,048 nodes, varying the cutoff to obtain mean receiver degrees from 4 to 1,023. For *graph-count scaling*, we repeat one 128-node, mean-degree-32 graph from $B = 2$ to 511, increasing total N from 256 to 65,408. Both axes span 8.2×10^3 to 2.1×10^6 edges, separating longer neighborhoods from proportional node-and-edge scaling. Both backends are measured until OOM; Appendix Table 4 gives the exact grids and realized degree statistics.

Representative irrep profiles. Input/output irreps are equal with alternating even/odd parity; spherical harmonics contain one copy per order through $L_{\max}$, and the edge embedding has width $K = 16$. The $L_{\max} = 1, 2, 3$ profiles are $128/64$, $128/64/64$, and $128/64/64/64$. Holding 128 scalar channels and 64 channels in every included non-scalar irrep ensures that angular arithmetic and radial-weight count rise together: $N_{\mathrm{w}} = 36{,}864$, $73{,}728$, and $131{,}072$.

For fp32 materialization, $M_W = 4|\mathcal{E}|N_{\mathrm{w}}$ bytes. Thus W alone reaches 32 GiB at approximately 2.3×10^5, 1.2×10^5, and 6.6×10^4 directed edges for $128/64$, $128/64/64$, and $128/64/64/64$, respectively. Inputs, outputs, library state, and temporary storage make the practical capacity lower.

5.2 Memory scaling

We first seek the practical memory scaling law: whether peak allocation follows the materialized $M_W \propto |\mathcal{E}|N_{\mathrm{w}}$ term as edge count and operator width increase, or whether SOBEK replaces that term with bounded workspace. Figure 2 compares peak allocated memory under both scaling constructions and all three differentiation orders. At the last degree point shared by both backends, OpenEquivariance allocates 24.1–27.1 GiB across the nine profile/pass pairs, while SOBEK allocates 0.1–0.4 GiB. OpenEquivariance's confirmed OOMs occur between 1.6×10^4 and 2.6×10^5 edges, depending on profile and differentiation order. Every SOBEK curve reaches the campaign endpoint of 2.1×10^6 edges, where peak allocation remains 0.4–1.5 GiB.

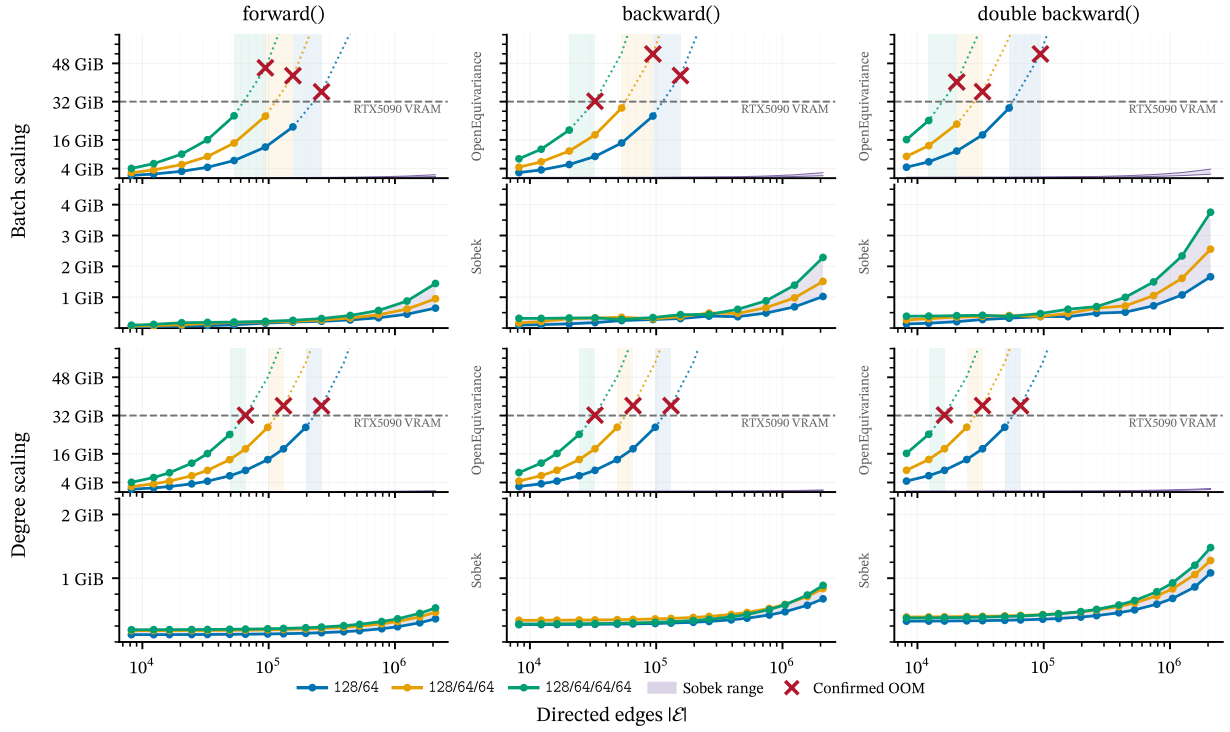


Figure 2: Peak allocated CUDA memory under graph-count (top) and degree (bottom) scaling, with differentiation order on columns. Each panel separates OpenEquivariance and SOBEK linear ranges. Profile colors identify multiplicity schedules; purple bands show SOBEK's cross-profile range and repeat it on the OpenEquivariance axes for scale. Dotted curves show M_W and the dashed line marks 32 GiB RTX 5090 capacity; shaded intervals end at confirmed OOMs (red crosses).

The result is not constant total memory. SOBEK removes the dominant $O(|\mathcal{E}|N_{\mathrm{w}})$ allocation by never materializing per-edge radial weights or edge messages, and caps tensor-product scratch at 128 MiB. The graph itself still requires edge indices, spherical harmonics, embeddings, and their saved gradients, all of which grow with $|\mathcal{E}|$; under graph-count scaling, node features, outputs, and node-scaled intermediates grow with N as well. Consequently, fixed-N degree scaling retains a much smaller residual edge-linear component, while graph-count scaling combines that component with node-linear storage. Operator width changes these constants, but no longer multiplies edge count by the full radial-weight count.

The observed law is therefore bounded tensor-product workspace plus unavoidable graph and feature storage, rather than $O(1)$ memory overall.

5.3 Throughput scaling

We next ask whether removing edge-weight materialization preserves processing rate as well as capacity. Graph-count scaling measures how throughput changes as larger workloads fill the GPU, while degree scaling tests whether longer receiver neighborhoods change per-edge execution efficiency. Figure 3 shows the scaling law directly. OpenEquivariance throughput is effectively constant with workload for each profile and pass; its degree curves overlap its graph-count curves and are therefore omitted from the figure. Every point completed by both backends was nevertheless measured and passed strict double-peer validation. At the last degree workload shared by both backends, SOBEK speedups span 3.8–54.4× across the nine profile/pass pairs. At the exact OpenEquivariance graph-count capacity, speedups across the three primary profiles are 19.7–42.1× for forward, 5.6–16.9× for backward, and 7.5–15.0× for double backward.

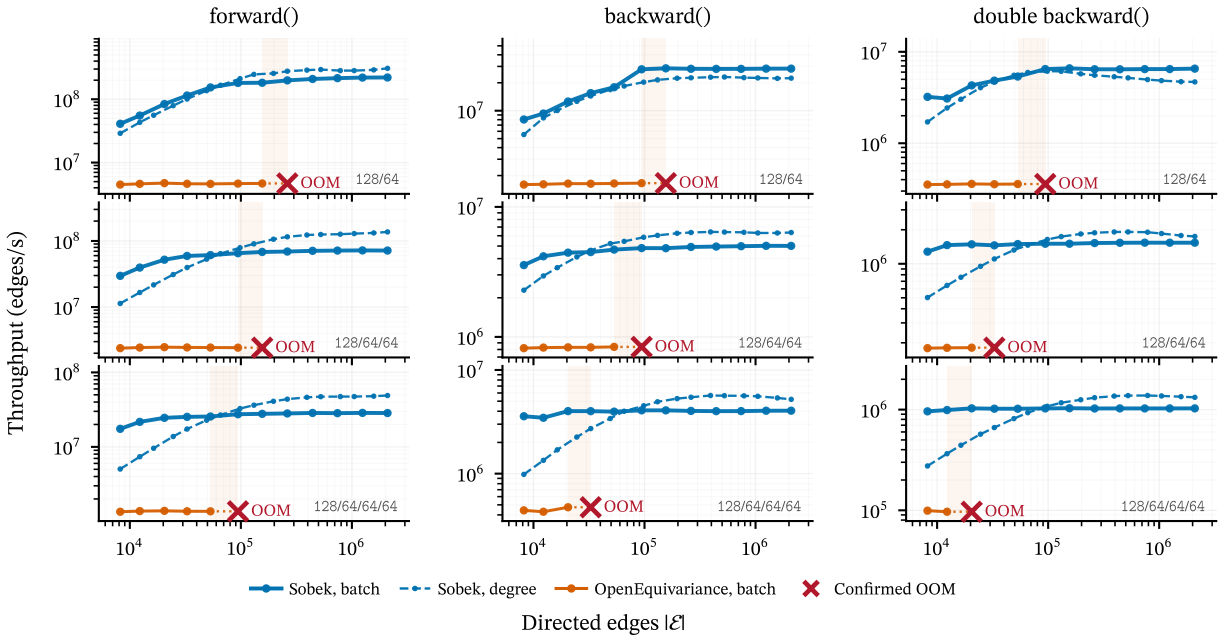


Figure 3: Absolute throughput by profile row, with differentiation order on columns. Solid and dashed blue curves show SOBEK graph-count and degree scaling; orange shows OpenEquivariance graph-count scaling. Both axes are logarithmic, with panel-specific throughput ranges. OpenEquivariance degree curves are omitted because they overlap graph-count curves. Orange bands and dotted segments connect the last success to confirmed OOMs (red crosses).

All SOBEK graph-count curves reach 2.1×10^6 edges and retain at least 99.2% of their peak throughput at that endpoint. Under degree scaling, forward peaks at the endpoint, backward retains 91.8–99.0% of peak, and double backward retains 76.3–95.9%. For the three primary profiles, extending graph-count scaling from the OpenEquivariance capacity to $B = 511$ increases work per call by 9.5–170.3×; SOBEK's maximum throughput over the extended curve is 1.0–1.2× its throughput at the matched capacity. The larger feasible workloads therefore primarily increase usable problem size while maintaining or modestly improving throughput.

This behavior follows directly from the avoided memory operations. A materialized `uvw` schedule computes N_w radial weights per edge, writes them to GPU memory, and reads them back for the contraction; N_w grows quadratically with multiplicity. SOBEK consumes these values without copying an $|\mathcal{E}| \times N_w$ tensor through global memory.

Removing those single-use writes and reads avoids OpenEquivariance's early capacity boundary and permits larger batches without increasing the bounded tensor-product workspace.

5.4 Generality across feature structures

We use capacity matching to compare varied representations fairly. For each profile, we independently determine for each pass the maximum batch size (graph count) B_{oeq} successfully completed by OpenEquivariance, then time both backends on exactly the same input and double-peer cross-validate their outputs to ensure the same numerical result under the strict policy above. This gives the largest workload that both OpenEquivariance and SOBEK can execute. These are OpenEquivariance-capacity-limited common workloads, not SOBEK throughput ceilings. Capacity matching therefore understates both usable problem size and throughput at larger, better-occupying batches.

The main campaign covers sixteen fully connected `uvw` profiles on the fixed graph. The grid varies angular order, channel width, scalar fraction, and radial-operator size. It tests whether SOBEK's gains persist as feature structure shifts from scalar-heavy to more evenly distributed representations, as higher-order irreps add angular arithmetic, and as wider operators increase channel mixing and radial-weight count. This breadth establishes generality beyond the three profiles chosen to expose the scaling laws.

Table 3: Capacity-matched `uvw` results on the fixed 128-node, degree-32 graph. Each pass uses OpenEquivariance's largest graph count B_{oeq}. Bold profiles are those used in the scaling-law figures. Paired Peak GiB and M edges/s cells use S | oeq order. Peak GiB is peak allocated CUDA memory. Profiles list multiplicities; N_{w} is the radial-weight count.

Profile	N_{w}	`forward()`				`backward()`				`double backward()`			
		Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq	Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq	Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq
32/16	2,304	862	0.7 | 30.7	661.0 | 79.9	**8.28**×	431	0.6 | 30.7	104.7 | 24.4	**4.29**×	215	0.7 | 30.6	24.3 | 6.1	**4.00**×
32/32	4,096	487	0.5 | 30.7	723.5 | 45.7	**15.84**×	243	0.6 | 30.7	91.9 | 11.7	**7.88**×	121	0.5 | 30.5	22.9 | 3.5	**6.54**×
32/16/16	4,608	433	0.5 | 30.8	188.3 | 41.7	**4.52**×	216	0.6 | 30.7	17.4 | 12.7	**1.37**×	108	0.5 | 30.6	5.3 | 3.1	**1.73**×
32/32/32	11,264	178	0.3 | 30.8	170.4 | 16.0	**10.62**×	88	0.4 | 30.4	11.9 | 5.1	**2.32**×	44	0.4 | 30.4	3.7 | 1.2	**3.00**×
32/16/16/16	8,192	244	0.4 | 30.7	85.9 | 23.3	**3.68**×	121	0.5 | 30.5	12.7 | 6.7	**1.89**×	60	0.4 | 30.2	3.3 | 1.7	**1.93**×
32/32/32/32	23,552	85	0.3 | 30.7	70.8 | 8.3	**8.48**×	42	0.3 | 30.3	6.9 | 2.7	**2.54**×	21	0.4 | 30.3	1.8 | 0.6	**2.99**×
64/32	9,216	217	0.3 | 30.7	423.9 | 20.9	**20.28**×	108	0.4 | 30.5	58.8 | 6.4	**9.14**×	54	0.4 | 30.5	14.3 | 1.6	**9.16**×
64/64	16,384	122	0.3 | 30.6	345.7 | 12.2	**28.31**×	61	0.3 | 30.6	43.3 | 3.7	**11.59**×	30	0.4 | 30.1	9.2 | 0.8	**10.83**×
64/32/32	18,432	108	0.3 | 30.5	125.7 | 10.7	**11.79**×	54	0.3 | 30.5	9.8 | 3.2	**3.03**×	27	0.4 | 30.5	3.1 | 0.7	**4.25**×
64/64/64	45,056	44	0.2 | 30.3	86.4 | 4.3	**19.94**×	22	0.3 | 30.4	5.5 | 1.4	**4.07**×	11	0.4 | 30.4	1.7 | 0.3	**5.82**×
64/32/32/32	32,768	61	0.2 | 30.6	59.7 | 5.3	**11.28**×	30	0.3 | 30.1	7.9 | 1.9	**4.20**×	15	0.4 | 30.1	2.0 | 0.4	**5.05**×
64/64/64/64	94,208	21	0.2 | 30.3	34.6 | 1.9	**18.04**×	10	0.3 | 28.9	3.2 | 0.7	**4.93**×	5	0.4 | 28.9	0.8 | 0.1	**5.79**×
128/64	36,864	54	0.2 | 30.5	197.0 | 4.7	**42.13**×	27	0.3 | 30.5	28.1 | 1.7	**16.95**×	13	0.3 | 29.4	5.4 | 0.4	**14.99**×
128/128	65,536	30	0.2 | 30.1	132.2 | 2.7	**49.68**×	15	0.3 | 30.1	14.0 | 1.0	**14.73**×	7	0.3 | 28.1	4.0 | 0.2	**19.87**×
128/64/64	73,728	27	0.2 | 30.4	66.8 | 2.5	**27.21**×	13	0.3 | 29.4	4.7 | 0.8	**5.60**×	6	0.4 | 27.1	1.4 | 0.2	**7.51**×
128/64/64/64	131,072	15	0.2 | 30.1	26.8 | 1.4	**19.71**×	7	0.3 | 28.1	4.0 | 0.5	**8.46**×	3	0.4 | 24.1	1.0 | 0.1	**10.24**×

Across the grid, SOBEK is faster in all 48 profile/pass comparisons, with geometric-mean speedups of $14.9\times$, $5.0\times$, and $5.7\times$ for forward, backward, and double backward. Individual comparisons range from $1.4\times$ to $49.7\times$. Its matched peak allocation is 0.2–0.7 GiB versus 24.1–30.8 GiB, and every point uses zero edge-weight workspace.

We repeat the same protocol for homogeneous channelwise `uvu`, using three equal-multiplicity widths at each angular order. SOBEK is faster in all 27 profile/pass comparisons. Geometric-mean speedups are $3.3\times$, $3.4\times$, and $1.8\times$ for forward, backward, and double backward, with respective ranges of 1.2–$5.9\times$, 2.5–$4.2\times$, and 1.6–$2.1\times$. Matched peak allocation is 1.7–4.5 GiB for SOBEK versus 30.3–30.8 GiB for OpenEquivariance, and every SOBEK point uses zero edge-weight workspace. Appendix Table 5 reports the complete grid.

The smaller `uvu` speedups are consistent with the same mechanism. Tied-channel mixing removes an output-multiplicity factor from the materialized edge-weight expansion, so streaming eliminates a smaller fraction of total work; higher angular order also raises the relative cost of Clebsch–Gordan contractions. The timing and peak-allocation results support this explanation based on avoided memory reads and writes but do not isolate it from occupancy, cache locality, launch overhead, or kernel mapping.

6 Related Work

Memory-efficient exact operators and streaming reformulations. Two close systems analogues are KeOps and FlashAttention. KeOps generates GPU map-reductions over symbolic pairwise arrays, including automatic differentiation, without materializing the complete kernel or distance matrix [16]. FlashAttention preserves exact attention while changing its evaluation order so that the dominant global intermediate is never materialized [17]. FlashAttention-2

further shows that work partitioning and hardware mapping remain important after the algorithmic memory transformation [18]. SOBEK applies the same separation between a streaming formulation and its hardware realization to irregular equivariant graph convolution: streaming determines which intermediates survive globally, while receiver factorization and generated CUDA determine how the resulting computation maps to the GPU.

Equivariant kernels and generated GPU systems. OpenEquivariance generates sparse, problem-specialized kernels for equivariant tensor products and deterministic graph convolutions [11]. NVIDIA cuEquivariance provides optimized CUDA execution for a general segmented-tensor-product representation [12], and recent work has studied its integration and reduced-precision execution in MACE [19]. FlashTP combines kernel fusion, Clebsch–Gordan sparsity, and path aggregation to reduce intermediate memory traffic in equivariant interatomic potentials [20]. SOBEK shares their use of static tensor-product descriptions and specialized kernels but places its main optimization boundary around the complete edge-conditioned convolution, where the lifetime of radial weights and their adjoints determines peak memory.

Alternative operators and connection modes. Other work reduces cost by changing the equivariant operator or its channel connectivity. eSCN reduces $\mathrm{SO}(3)$ convolutions to operations in $\mathrm{SO}(2)$, and EquiformerV2 uses this construction for higher-degree representations [21, 13]. Equiformer and InfGCN instead use depthwise or channelwise tensor products to avoid fully connected multiplicity mixing within the tensor product [8, 22]. Xie et al. [10] formalize the resulting distinction between arithmetic scaling and the equivariant bilinear maps an operator can represent. These choices can reduce arithmetic or parameter cost, but they do not by themselves eliminate replication over graph edges in a materialized schedule. SOBEK is orthogonal: it provides native `uvu` execution for tied-channel models while its streamed `uvw` path retains fully connected couplings without the $|\mathcal{E}|$-scaled weight intermediate.

7 Conclusion

The central result of this work is that the large global intermediates commonly associated with edge-conditioned equivariant convolutions are consequences of their evaluation schedule, not requirements of the operator. Forward, first backward, and double backward can instead be organized as streamed edge-local products consumed immediately by receiver-, source-, or edge-owned reductions. This reformulation preserves fully connected multiplicity mixing and the exact derivative structure while replacing edge-scaled workspace with bounded receiver-side state and recomputation.

SOBEK realizes this reformulation as a generated-CUDA backend. Its receiver-factorized schedules evaluate radial-to-CG projections without materializing the edge-wise weight and message tensors that dominate memory in dynamic molecular graphs. The implementation supports forward, first backward, and double backward with bounded auxiliary state and without constructing the edge-weight adjoint.

Extending the same bounded-lifetime principle to eSCN-style edge-aligned $\mathrm{SO}(2)$ kernels is a particularly promising direction, combining their lower angular complexity with streaming execution at higher angular resolution [21].

By separating operator expressivity from edge-sized workspace, streaming execution relaxes the tradeoffs among neighborhood size, representation width, batch size, and channel connectivity. This could make it practical to run full-atom machine-learning interatomic potentials on proteins and other large biomolecular assemblies, use longer-range or higher-resolution representations without collapsing batch size, and train or sample large equivariant generative models over substantially larger structures. More broadly, it would let model designers choose representation width and connectivity for physical fidelity rather than for the peak memory of a transient edge tensor. We hope this systems result translates into broader capabilities throughout the geometric deep learning and molecular modeling ecosystem.

Appendix A Benchmark Regimes

Table 4: Degree and graph-count parameter grids. Degree statistics are realized over receiver nodes, with mean $\bar{d}$ and standard deviation σ_d; N is exact and $|\mathcal{E}|$ is rounded to two significant figures.

(a) Degree: vary neighborhood size
Fixed: $B = 1$
Nodes/graph = total $N = 2{,}048$

$\bar{d}$	σ_d	$\lvert\mathcal{E}\rvert$	$\bar{d}$	σ_d	$\lvert\mathcal{E}\rvert$
4	1.95	8.2×10^3	96	26.81	2.0×10^5
6	2.50	1.2×10^4	128	36.47	2.6×10^5
8	3.01	1.6×10^4	192	56.28	3.9×10^5
12	3.89	2.5×10^4	256	76.72	5.2×10^5
16	4.76	3.3×10^4	384	117.76	7.9×10^5
24	6.61	4.9×10^4	512	157.90	1.0×10^6
32	8.59	6.6×10^4	768	226.61	1.6×10^6
48	12.83	9.8×10^4	1,023	273.19	2.1×10^6
64	17.30	1.3×10^5			

(b) Graph count: vary B
Fixed: nodes/graph $= 128$
$\bar{d} = 32$; $\sigma_d = 10.49$

B	Total N	$\lvert\mathcal{E}\rvert$
2	256	8.2×10^3
3	384	1.2×10^4
5	640	2.0×10^4
8	1,024	3.3×10^4
13	1,664	5.3×10^4
23	2,944	9.4×10^4
38	4,864	1.6×10^5
64	8,192	2.6×10^5
108	13,824	4.4×10^5
181	23,168	7.4×10^5
304	38,912	1.2×10^6
511	65,408	2.1×10^6

Appendix B Additional Implementation Details

Appendix B.1 Static compile-time decisions

For a fixed irrep/path contract, radial width, derivative signature, GEMM mode, and scratch bound, the generated translation unit is invariant to graph shape. Node and edge counts, degree distribution, CSR ordering, retained state, and chunk schedule select only launches already present in the cached extension.

The remaining CUDA choices are resolved before source emission. Radial inputs use aligned `float4` loads when their width is divisible by four. If a `uvw` contract contains an output group of dimension at least five, groups of dimension at least three are split into kernels covering at most four output components. Backward source paths use a statically selected warp, 8-lane subwarp, or cooperative CTA executor; the CTA form is emitted only when its shared tile is at most 24 KiB. Edge derivatives use either one 32-lane owner or two 16-lane owners. Native `uvu` double backward additionally pairs compatible source paths and consecutive parameter-gradient groups; at component dimension seven and above, singleton directional-forward kernels contain at most four paths. These decisions fix path grouping and launch geometry without graph-dependent kernel dispatch.

Appendix B.2 Runtime scratch-budget policy

The configured budget B—128 MiB in all reported experiments—bounds floating receiver-side state, not graph indices, inputs, or outputs. In `uvw` forward, a group with component dimension d_g, receiver-state width D_g, and output multiplicity C_g processes at most

$$n_g = \min\left(|\mathcal{V}|, \max\left(1, \left\lfloor \frac{B}{4d_g(D_g + C_g)} \right\rfloor\right)\right) \tag{40}$$

receivers at once for fp32 budget B. When radial-parameter gradients are needed, `uvw` retains the fitting subset of complete groups that maximizes avoided sparse-CG recomputation, breaking ties by byte cost; `uvu` retains all groups only when their combined state fits. Other groups are recomputed over bounded receiver chunks.

On first-backward overflow, contracts with at least twenty paths may share one all-group node chunk; other contracts chunk groups independently. A chunked route is used only when it produces at least three chunks. Its source-CSR views and exact widths are prepared with the topology and reused by double backward. The second-order planner derives required state from active cotangents, greedily packs contiguous groups while retained state plus the largest temporary fits B, then chunks receivers. Forward-retained primal state is reused only when it enables one all-group, all-receiver chunk. Receiver and source chunks are independent, so source kernels traverse only prepared subchunks overlapping the active receiver interval. Graph growth therefore changes chunk counts and launches without introducing edge-scaled floating workspace.

Appendix C Native Channelwise Throughput

Table 5: Native channelwise uvu capacity-matched results on the fixed 128-node, degree-32 graph. Each pass uses OpenEquivariance's largest graph count B_{oeq}. Paired Peak GiB and M edges/s cells use S | oeq order. Peak GiB is peak allocated CUDA memory. Profiles list multiplicities; N_{w} is the radial-weight count.

Profile	N_{w}	`forward()`				`backward()`				`double backward()`			
		Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq	Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq	Graphs B_{oeq}	Peak GiB S | oeq	M edges/s S | oeq	Ratio S/oeq
64/64/64/64	1,472	1279	3.4 | 30.8	108.1 | 90.7	**1.19**×	641	2.7 | 30.6	28.3 | 9.7	**2.91**×	323	2.1 | 30.5	8.6 | 5.4	**1.58**×
128/128/128	1,408	1332	3.7 | 30.8	390.7 | 86.7	**4.51**×	669	3.0 | 30.7	41.0 | 9.9	**4.15**×	337	2.3 | 30.6	9.9 | 5.7	**1.74**×
128/128/128/128	2,944	647	3.0 | 30.8	57.5 | 39.5	**1.46**×	323	2.5 | 30.6	14.7 | 5.1	**2.87**×	163	1.9 | 30.6	4.3 | 2.5	**1.73**×
256/256	1,024	1807	4.5 | 30.8	513.8 | 129.1	**3.98**×	909	3.6 | 30.6	52.5 | 12.5	**4.20**×	460	2.8 | 30.6	14.9 | 8.0	**1.86**×
256/256/256	2,816	672	3.4 | 30.7	173.4 | 37.0	**4.69**×	336	2.8 | 30.6	20.2 | 5.3	**3.78**×	169	2.1 | 30.5	5.0 | 2.6	**1.88**×
256/256/256/256	5,888	325	2.8 | 30.7	28.9 | 13.9	**2.07**×	162	2.4 | 30.6	6.9 | 2.8	**2.50**×	81	1.7 | 30.3	2.0 | 1.2	**1.74**×
512/512	2,048	914	4.1 | 30.8	262.2 | 52.7	**4.98**×	458	3.4 | 30.7	27.4 | 8.6	**3.17**×	231	2.5 | 30.5	7.8 | 3.9	**2.00**×
512/512/512	5,632	338	3.2 | 30.8	90.5 | 15.4	**5.88**×	168	2.7 | 30.5	10.6 | 3.0	**3.51**×	85	2.0 | 30.5	2.6 | 1.3	**1.96**×
1024/1024	4,096	459	3.8 | 30.7	141.1 | 25.8	**5.46**×	229	3.3 | 30.6	15.0 | 4.1	**3.66**×	116	2.4 | 30.6	4.2 | 2.0	**2.10**×

References

[1] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

[2] Risi Kondor, Zhen Lin, and Shubhendu Trivedi. Clebsch–gordan nets: A fully fourier space spherical convolutional neural network. In *Advances in Neural Information Processing Systems*, volume 31, pages 10138–10147, 2018.

[3] Fabian B. Fuchs, Daniel E. Worrall, Volker Fischer, and Max Welling. SE(3)-transformers: 3d roto-translation equivariant attention networks. In *Advances in Neural Information Processing Systems*, volume 33, pages 1970–1981, 2020.

[4] Johannes Brandstetter, Rob Hesselink, Elise van der Pol, Erik J. Bekkers, and Max Welling. Geometric and physical quantities improve E(3) equivariant message passing. In *International Conference on Learning Representations*, 2022.

[5] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds, 2018.

[6] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P. Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E. Smidt, and Boris Kozinsky. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1):2453, 2022.

[7] Ilyes Batatia, Dávid P. Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. In *Advances in Neural Information Processing Systems*, volume 35, pages 11423–11436, 2022.

[8] Yi-Lun Liao and Tess Smidt. Equiformer: Equivariant graph attention transformer for 3d atomistic graphs. In *International Conference on Learning Representations*, 2023.

[9] Albert Musaelian, Simon Batzner, Anders Johansson, Lixin Sun, Cameron J. Owen, Mordechai Kornbluth, and Boris Kozinsky. Learning local equivariant representations for large-scale atomistic dynamics. *Nature Communications*, 14(1):579, 2023.

[10] Yuqing Xie, Ameya Daigavane, Mit Kotak, and Tess Smidt. The price of freedom: Exploring expressivity and runtime tradeoffs in equivariant tensor products. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 68599–68625. PMLR, 2025.

[11] Vivek Bharadwaj, Austin Glover, Aydın Buluç, and James Demmel. An efficient sparse kernel generator for O(3)-equivariant deep networks. In *Proceedings of the 2025 SIAM Conference on Applied and Computational Discrete Algorithms (ACDA)*, pages 32–46. Society for Industrial and Applied Mathematics, 2025.

[12] NVIDIA. cuEquivariance. Software, 2025. CUDA library for equivariant neural networks.

[13] Yi-Lun Liao, Brandon M. Wood, Abhishek Das, and Tess Smidt. EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations. In *International Conference on Learning Representations*, 2024.

[14] Mario Geiger and Tess Smidt. e3nn: Euclidean neural networks. *arXiv preprint arXiv:2207.09453*, 2022.

[15] e3nn developers. Tensor product. e3nn documentation, 2026. Accessed 2026-07-11.

[16] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the GPU, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.

[18] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.

[19] Alexandre Benoit. Speeding up MACE: Low-precision tricks for equivarient force fields, 2025.

[20] Seung Yul Lee, Hojoon Kim, Yutack Park, Dawoon Jeong, Seungwu Han, Yeonhong Park, and Jae W. Lee. FlashTP: Fused, sparsity-aware tensor product for machine learning interatomic potentials. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 33143–33156. PMLR, 2025.

[21] Saro Passaro and C. Lawrence Zitnick. Reducing SO(3) convolutions to SO(2) for efficient equivariant GNNs. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 27420–27438. PMLR, 2023.

[22] Chaoran Cheng and Jian Peng. Equivariant neural operator learning with graphon convolution. In *Advances in Neural Information Processing Systems*, volume 36, 2023.